How-To Library

Informatica

Measuring sales effectiveness with Amazon SageMaker Lakehouse

© Copyright Informatica LLC 2025, 2026. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

You can use Amazon SageMaker Lakehouse Connector to access Apace Iceberg's time travel capabilities on data stored in Amazon S3 and managed by the AWS Glue Data Catalog or S3 tables catalog within Amazon SageMaker Lakehouse.

Supported Versions

- Cloud Data Integration

Table of Contents

Use case overview. 2
Configuring mappings to access sales data ingestion and time travel in Apache Iceberg. 3
Before you begin. 4
Configure a Source Transformation. 8
Configure a Target Transformation. 10
Conclusion. 12

Use case overview

RetailRush is a prominent e-commerce company that conducts large-scale sales events during peak seasons such as Christmas and Black Friday.

These events generate vast amounts of transactional data involving frequent price changes, flash discounts, and real-time product updates across systems.

Existing challenges

RetailRush encounters multiple challenges stemming from high transaction volumes, ongoing data corrections, and delayed event arrivals during their large sales events. The following critical issues impact RetailRush's ability to effectively analyze and manage their sales data:

- Maintaining accurate historical snapshots of sales data for precise analysis
- Investigating anomalies such as abrupt sales drops with the correct data context
- Comparing current event performance with previous years to identify trends
- Meeting audit requirements by reconstructing the exact data state during the sales window

RetailRush also encountered a bug that caused incorrect price updates during sales between 6:00 PM and 7:00 PM. While this issue has been resolved and corrected within the data pipeline, the finance team needs a detailed analysis to accurately determine its effect on revenue metrics.

Solution

RetailRush can implement a solution using Amazon SageMaker Lakehouse Connector that leverages Apache Iceberg's time travel capabilities and thereby overcome their challenges:

- Sales transactions can be ingested and updated hourly into Apache Iceberg tables stored in Amazon S3, with AWS Glue Catalog managing metadata.
- As each data update creates an incremental Iceberg snapshot while preserving historical data states, analysts and applications can query any previous snapshot through Apache Iceberg's time travel feature. This enables

correct analysis of past sales states for performance comparison, anomaly detection, auditing, and incident impact assessment.

- If revenue data snapshots are retained for periods impacted by bugs, finance analysts can use them to accurately quantify the pricing bug's impact on reported revenue, eliminating the need for manual backups or data restoration.

Business benefits

RetailRush can realize significant business benefits from this solution:

- Establish a single source of truth for historical and current sales data without redundant copies.
- Enable reliable comparative analyses and anomaly investigations based on accurate historical snapshots.
- Support auditing and compliance by allowing recreation of sales data state as it existed during critical periods.
- Facilitate rapid investigation of pricing or data quality incidents, quantifying their financial impact through direct snapshot comparisons.
- Enhance decision-making with timely, precise insights into sales event performance and issue resolution impact.

Technical capabilities driving business benefits

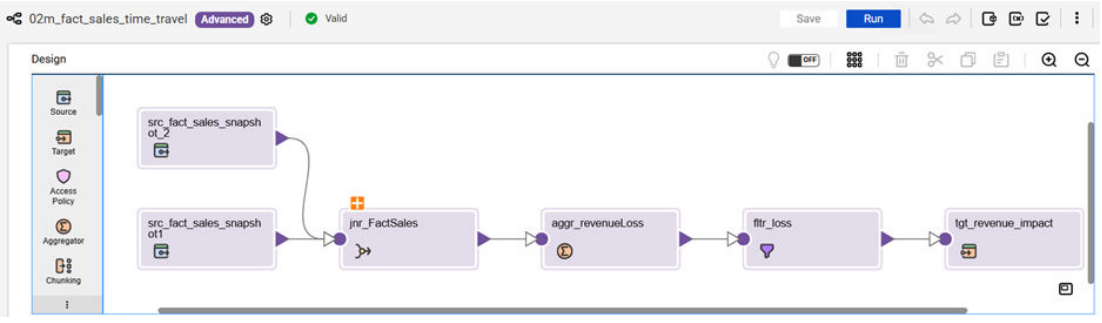
The following technical capabilities drive the desired business benefits:

- A centralized data architecture with version-controlled storage eliminates data duplication and ensures data consistency.
- Indexing and efficient storage of historical data versions support fast comparative queries.
- Real-time query and analysis infrastructure supports immediate financial impact calculations.
- Low-latency data ingestion and processing pipelines provide up-to-date information.

Configuring mappings to access sales data ingestion and time travel in Apache Iceberg

To ensure seamless ingestion and updates of sales transaction data into Apache Iceberg tables, RetailRush can configure Amazon SageMaker Lakehouse mappings that leverage the Apache Iceberg time travel capabilities.

The following image shows a sample mapping named *m_fact_sales_time_travel* that is designed to maintain accurate snapshots of sales data from Iceberg tables by querying specific snapshots in time:



The mapping uses an Amazon SageMaker Lakehouse connection to read data from Apache Iceberg tables stored in Amazon S3, with metadata managed by AWS Glue Data Catalog.

RetailRush can apply multiple transformations in the mapping to integrate and enrich data from various Iceberg tables such as Sales Transactions, Pricing Updates, and Product Catalogs.

The mapping supports incremental updates and corrections by leveraging snapshot identifiers or timestamps. This ensures precise extraction of data and reflects the state of the sale at any required moment, which is especially critical for investigations such as pricing bug impact analysis between 6:00 PM and 7:00 PM.

Finally, RetailRush can write the processed and enriched data into Iceberg tables within Amazon SageMaker Lakehouse through an Amazon SageMaker Lakehouse connection.

Before you begin

Before you configure a mapping, ensure that you meet the following prerequisites:

- Create IAM policies with the minimum required permissions to interact with Apache Iceberg tables managed by AWS Glue Catalog or S3 tables catalog.
- To use EC2 role to assume role authentication for Amazon S3, configure the EC2 role to assume another IAM role specified by the IAM Role ARN.
- Create an Amazon SageMaker Lakehouse connection. For information about configuring an Amazon SageMaker Lakehouse connection, see Connect to Amazon SageMaker Lakehouse.
- If the source and target are located in different regions, create an Amazon S3 bucket ARN. For more information about creating ARN for an Amazon S3 bucket, see "Prerequisites for mappings" on page 7.

Create minimal IAM policies

You need to create IAM policies with the minimum required permissions to interact with Apache Iceberg tables managed by AWS Glue Catalog or S3 tables catalog. For more information on configuring these policies, refer to the AWS documentation.

Minimum policy for Amazon Athena

The following sample policy shows the minimal Amazon IAM policy to access Amazon Athena:

```
{
    "Version": "2025-10-17",
    "Statement": [
        {
            "Sid": "VisualEditor0",
            "Effect": "Allow",
            "Action": [
                "athena:CreatePreparedStatement",
                "athena:GetPreparedStatement",
                "athena:GetWorkGroup",
                "athena:GetTableMetadata",
                "athena:StartQueryExecution",
                "athena:GetQueryResultsStream",
                "athena:ListDatabases",
                "athena:GetQueryExecution",
                "athena:GetQueryResults",
                "athena:GetDatabase",
                "athena:ListTableMetadata",
                "athena:GetDataCatalog",
                "athena:DeletePreparedStatement"
            ],
            "Resource": [
                "arn:aws:athena:*:*:workgroup/*",
                "arn:aws:athena:*:*:datacatalog/*"
            ]
        },
        {
            "Sid": "VisualEditor1",
            "Effect": "Allow",
            "Action": [
                "athena:ListDataCatalogs",
                "athena:GetQueryExecution",
                "athena:ListWorkGroups",
```

```
                "athena:GetPreparedStatement"
            ],
            "Resource": "*"
        }
    ]
}
```

Minimum policy for AWS Glue

The following sample policy shows the minimal Amazon IAM policy to access AWS Glue Catalog:

```
{
    "Version": "2025-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "glue:*"
            ],
            "Resource": [
                "*"
            ]
        }
    ]
}
```

Minimum policy for Amazon S3

The following sample policy shows the minimal Amazon IAM policy to read from or write data to an Amazon S3 bucket:

```
{
    "Version": "2025-10-17",
    "Statement": [
        {
            "Sid": "VisualEditor0",
            "Effect": "Allow",
            "Action": [
                "s3:PutObject",
                "s3:GetObject",
                "s3:ListBucket",
                "s3:DeleteObject"
            ],
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation",
                "s3:ListAllMyBuckets",
                "s3:GetBucketAcl"
            ],
            "Resource": [
                "*"
            ]
        }
    ]
}
```

Minimum policy for S3 tables

The following sample policy shows the minimal Amazon IAM policy to read from or write data to S3 tables:

```
{
    "Version": "2025-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "sts:AssumeRole",
            "Resource": "arn:aws:iam::001234567890:role/S3TableSparkrole"
```

```
            },
            {
                "Effect": "Allow",
                "Action": [
                    "s3tables:CreateTable",
                    "s3tables:GetTable",
                    "s3tables:ListTables",
                    "s3tables:DeleteTable",
                    "s3tables:GetTableMetadataLocation",
                    "s3tables:GetTableData",
                    "s3tables:PutTableData",
                    "s3tables:UpdateTableMetadataLocation"
                ],
                "Resource": [
                    "arn:aws:s3tables:us-east-1:001234567890:bucket/sagemaker-s3tables",
                    "arn:aws:s3tables:us-east-1:001234567890:bucket/sagemaker-s3tables/*",
                    "arn:aws:s3tables:us-east-1:001234567890:bucket/sagemaker-s3tables/
table/*"
                ]
            }
        ]
    }
```

Minimum policy for AWS Lake Formation

The following sample policy shows the minimal Amazon IAM policy to access AWS Lake Formation:

```
{
    "Version": "2025-10-17",
    "Statement": [
        {
            "Sid": "VisualEditor0",
            "Effect": "Allow",
            "Action": "lakeformation:*",
            "Resource": "*"
        }
    ]
}
```

Configure EC2 role to assume role

You can configure an EC2 role to assume an IAM role and generate temporary security credentials to connect to Amazon S3 from the same or different AWS accounts.

When you configure an EC2 role to assume an IAM role, ensure that you have the **sts:AssumeRole** permission and a trust relationship established within the AWS accounts to use temporary security credentials. The trust relationship is defined in the trust policy of the IAM role when you create the role. The IAM role adds the EC2 role as a trusted entity allowing the EC2 role to use the temporary security credentials and access the AWS accounts.

When the trusted EC2 role requests for the temporary security credentials, the AWS Security Token Service (AWS STS) dynamically generates the temporary security credentials that are valid for a specified period and provides the credentials to the trusted EC2 role.

Before you use the EC2 Role to Assume Role authentication, consider the following prerequisites:

- Install the Secure Agent on the AWS EC2 instance.
- The EC2 role attached to the AWS EC2 instance must have permissions to assume another IAM role.
  The following code snippet a sample permission policy of EC2 role that is attached to the AWS EC2 instance:

```
{
    "Version": "2025-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "sts:AssumeRole",
            "Resource": "arn:aws:iam::001234567890:role/sagemaker_rolearn"
```

```
            }
        ]
    }
```

Resource value must include the ARN of IAM role that the EC2 role needs to assume.

- The IAM role that the EC2 role needs to assume must have a permission policy and a trust policy attached to access AWS Glue Catalog, Amazon Athena, and Amazon S3.
  You can also specify the external ID of your AWS account for a more secure access. The external ID must be a string.

  The following code snippet shows the assumed IAM role's trust policy with the external ID:

```
{
    "Version": "2025-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::001234567890:root" //anyone in the account
001234567890 can assume this role, this can also be limited to one role.
            },
            "Action": "sts:AssumeRole",
            "Condition": {
                "StringEquals": {
                    "sts:ExternalId": "aws_externalid"
                }
            }
        }
    ]
}
```

Prerequisites for mappings

Before you configure and run a mapping, complete the required prerequisites.

When you read from and write to Apache Iceberg tables, you can specify the Iceberg Spark properties to configure the Spark behavior for the Iceberg tables at runtime.

When the source and target are located in different regions, you must specify the Amazon S3 bucket ARN property in the Iceberg Spark properties in the Advanced source and target properties.

Create ARN for Amazon S3 bucket

To create the Amazon S3 bucket ARN, perform the following steps:

1. Sign in to the AWS Management Console and open the Amazon S3 console.
2. In the navigation bar, select the AWS region where you want to create an access point for the Amazon S3 bucket.
3. In the left navigation pane, select **Access Points**.
4. Click **Create access point**.
5. Enter a unique name for the access point.
6. Select the Amazon S3 bucket that the access point is associated with.
7. In the **Network origin** field, select **Internet**.
8. Click **Create access point**.
9. After the access point is created, click on the name to view the properties.
10. Copy the Amazon Resource Name (ARN) property value.

Configure a Source Transformation

Configure a Source transformation that uses an Amazon SageMaker Lakehouse connection to read data from Apache Iceberg tables stored in Amazon S3, with metadata managed by AWS Glue Data Catalog.

Sources for Amazon SageMaker Lakehouse

When you configure a mapping in advanced mode to use an Amazon SageMaker Lakehouse source, you can configure the source properties.

Specify the name and description of the Amazon SageMaker Lakehouse source. Configure the source and advanced source properties for the Amazon SageMaker Lakehouse object.

The following table describes the Amazon SageMaker Lakehouse source properties that you can configure in a Source transformation:

Property	Description
Connection	Name of the source connection. You can select an existing connection, create a new connection, or define parameter values for the source connection property. If you want to overwrite the source connection properties at run time, select the **Allow parameter to be overridden at run time** option.
Source Type	Type of the Amazon SageMaker Lakehouse source object. You can choose from the following source types: - Single Object. Select to specify a single Amazon SageMaker Lakehouse object. - Parameter. Select to specify a parameter name. You can configure the source object in a mapping task associated with a mapping that uses this Source transformation.
Parameter	A parameter file where you define values that you want to update without the need to edit the task. Select an existing parameter for the source object or click **New Parameter** to define a new parameter for the source object. The **Parameter** property appears only if you select parameter as the source type. If you want to overwrite the parameter at run time, select the **Allow parameter to be overridden at run time** option. When the task runs, the Secure Agent uses the parameters from the file that you specify in the advanced session properties.
Object	Source object for the mapping.

The following table describes the Amazon SageMaker Lakehouse query options that you can configure in a Source transformation:

Property	Description
Filter	Filters records based on the filter condition. Click **Configure** to add conditions to filter records and reduce the number of rows that the Secure Agent reads from the source. You can specify the following filter conditions: - Not Parameterized. Use a basic filter to specify the object, field, operator, and value to select specific records. - Completely Parameterized. Use a parameter to represent the field mapping. - Advanced. Use an advanced filter to define a more complex filter condition that uses the Amazon SageMaker Lakehouse query format. A mapping fails at run time when you specify a simple filter with `Starts With` operator to retrieve data. To avoid this issue, configure an advanced filter with a `like` query to filter query results based on patterns instead of exact textual matches.
Sort	Not applicable.

The following table describes the Amazon SageMaker Lakehouse source advanced properties that you can configure in a Source transformation:

Property	Description

Iceberg Spark Properties	The Spark configuration properties as key-value pairs that you want to configure for the Iceberg tables at runtime.
	Enter the properties in the following format:
	`<parameter name>=<parameter value>`
	If you enter more than one property, enter each property in a new line.
	When you use the S3 Tables lakehouse pattern, you must specify the S3 table bucket ARN property in the following format:
	`TableBucketARN=arn:aws:s3tables:us-east-1:001234567890:bucket/sagemaker-s3tables`
	When the source and target are in different regions, you must specify the Amazon S3 bucket ARN property in the following format:
	`s3.access-points.<bucket name>=<S3 bucket ARN>`
	For more information about how to create the ARN for S3 bucket, see "Prerequisites for mappings" on page 7.

Pre-SQL	The SQL queries to run before reading data from Apache Iceberg tables. Ensure that the SQL queries use a valid Spark SQL syntax. You can enter multiple queries separated by a semicolon. You must prefix the table name with the `<LakehouseCatalog>` string and the database name. For example, `<LakehouseCatalog>.databasename.tablename` The database name and table name identifiers in the queries must be in lowercase. If the pre-SQL query fails, the mapping also fails.

Property	Description
Post-SQL	The SQL queries to run after reading data from Apache Iceberg tables. Ensure that the SQL queries use a valid Spark SQL syntax. You can enter multiple queries separated by a semicolon. You must prefix the table name with the `<LakehouseCatalog>` string and the database name. For example, `<LakehouseCatalog>.databasename.tablename` The database name and table name identifiers in the queries must be in lowercase. If the post-SQL query fails, the mapping also fails. If the mapping fails, the post-SQL query is not executed.
Tracing Level	Sets the amount of detail that appears in the log file. You can choose terse, normal, verbose initialization, or verbose data. Default is normal.

Using time travel queries

You can leverage time travel queries to audit data from a specific snapshot timeframe without the need to manually maintain duplicate datasets.

This approach ensures data integrity and simplifies compliance reporting by allowing access to data exactly as it existed at a particular point or interval in the past.

When you configure a source transformation in a mapping in advanced mode, you can use the advanced filter option in your Amazon SageMaker Lakehouse object to configure a time travel query. The query fetches the data based on a timestamp or snapshot ID you specify.

You can use one of the following queries to retrieve the data:

- **Query by Timestamp**: You can query an Iceberg table as it existed at a particular timestamp.
  For example, if you want to time travel to July 10, 1986 at 04:20:09, use the following time travel query:

  ```
  TimestampValue = '1986-07-10 04:20:09.0 UTC'
  ```

- **Query by Snapshot ID**: You can query an Iceberg table by specifying a snapshot ID. Each snapshot has a unique identifier.
  For example, if you want to time travel to a snapshot with ID 33444444553321, use the following time travel query:

  ```
  SnapshotVersion = 33444444553321
  ```

Configure a Target Transformation

Configure a Target transformation that uses an Amazon SageMaker Lakehouse connection and writes the processed and enriched data back into an Iceberg table for further analysis or reporting.

Targets for Amazon SageMaker Lakehouse

When you configure a mapping in advanced mode to use an Amazon SageMaker Lakehouse target, configure the target properties.

Specify the name and description of the Amazon SageMaker Lakehouse target. Configure the target and advanced target properties for the Amazon SageMaker Lakehouse object in a Target transformation.

The following table describes the properties that you can configure in a Target transformation:

Property	Description
Connection	Name of the target connection. You can select an existing connection, create a new connection, or define parameter values for the target connection property. If you want to overwrite the target connection properties at run time, select the **Allow parameter to be overridden at run time** option.
Target Type	Type of the Amazon SageMaker Lakehouse target object. You can choose from the following target types: - Single Object. Select to specify a single Amazon SageMaker Lakehouse object. - Parameter. Select to specify a parameter name. You can configure the target object in a mapping task associated with a mapping that uses this Target transformation.
Parameter	A parameter file where you define values that you want to update without having to edit the task. Select an existing parameter for the target object or click **New Parameter** to define a new parameter for the target object. The **Parameter** property appears only if you select parameter as the target type. If you want to overwrite the target object at run time, select the **Allow parameter to be overridden at run time** option. When the task runs, the Secure Agent uses the parameters from the file that you specify in the advanced session properties.
Object	Name of the target object. You can select an existing object from the list or create a target object at run time.
Operation	Type of the target operation. Select one of the following operations: - Insert - Update - Upsert - Delete You cannot use update, upsert, and delete target operations for S3 Tables lakehouse pattern. **Note:** You cannot configure the Data Driven operation on an Amazon SageMaker Lakehouse target.
Update Columns	The primary key columns to update, upsert, or delete data in a Amazon SageMaker Lakehouse target.

The following table describes the advanced properties that you can configure in a Target transformation:

Property	Description
Truncate Target	Truncates the target table before loading the data. By default, the property is not selected.

Iceberg Spark Properties	The Spark configuration properties as key-value pairs that you want to configure for the Iceberg tables at runtime.
	Enter the properties in the following format:
	`<parameter name>=<parameter value>`
	If you enter more than one property, enter each property in a new line.
	When you use the S3 Tables lakehouse pattern, you must specify the S3 table bucket ARN property in the following format:
	`TableBucketARN=arn:aws:s3tables:us-east-1:001234567890:bucket/sagemaker-s3tables`
	When the source and target are in different regions, you must specify the bucket region property in the following format for the update, upsert, or delete target operations:
	`BucketRegion=<Amazon-S3-target-bucket-region-name>`
	For more information about how to create the ARN for S3 bucket, see "Prerequisites for mappings" on page 7.

Update Mode	Determines how the records are updated in the target table. This property applies when you select the **Update** or **Upsert** target operation. Select one of the following modes: - Update As Update. Updates records in the target table if the specified unique key column value matches with the incoming column value. - Update Else Insert. Updates records in the target table if the specified unique key column value matches with the incoming column value. If the unique key column value does not match, the mapping inserts a new row with the records.
Pre-SQL	The SQL queries to run before writing data to Apache Iceberg tables. Ensure that the SQL queries use a valid Spark SQL syntax. You can enter multiple queries separated by a semicolon. You must prefix the table name with the `<OpenTableCatalog>` string and the database name. For example, `<OpenTableCatalog>.databasename.tablename` The database name and table name identifiers in the queries must be in lowercase. If the pre-SQL query fails, the mapping also fails.
Post-SQL	Post-SQL queries to run after writing data to Apache Iceberg tables. Ensure that the SQL queries use a valid Spark SQL syntax. You can enter multiple queries separated by a semicolon. You must prefix the table name with the `<OpenTableCatalog>` string and the database name. For example, `<OpenTableCatalog>.databasename.tablename` The database name and table name identifiers in the queries must be in lowercase. If the post-SQL query fails, the mapping also fails. If the mapping fails, the post-SQL query is not executed.

Conclusion

Amazon SageMaker Lakehouse Connector enables RetailRush to leverage Apache Iceberg's time travel capabilities through Cloud Data Integration, transforming how they analyze sales data during critical business events.

By implementing this solution, RetailRush can maintain accurate historical snapshots, investigate anomalies with proper context, and meet compliance requirements without maintaining redundant data copies.

Author

Informatica Documentation Team